Kubernetes networking deep dive

The network model, CNI plugins, Services and kube-proxy, EndpointSlices, CoreDNS, network policies, Ingress and the Gateway API, and a fixed method for debugging.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The Kubernetes network model: pods, nodes and the CNI (12 min)
2. CNI plugins: overlays, routing and eBPF (12 min)
3. Services: ClusterIP, NodePort and kube-proxy modes (12 min)
4. EndpointSlices and how traffic finds pods (11 min)
5. CoreDNS and service discovery (12 min)
6. Network policies in depth (13 min)
7. Ingress and the Gateway API (12 min)
8. Debugging Kubernetes networking (13 min)
Cheat sheet

Lesson 1. The Kubernetes network model: pods, nodes and the CNI

12 minutes. State the four rules of the Kubernetes network model and trace how a pod gets an address and a route through the CNI.

Kubernetes does not implement networking itself. It defines rules that any network must satisfy and delegates the work to a plugin. Knowing the rules tells you what should be possible, which is what you need when something is not.

What you will learn

- The rules every Kubernetes network must satisfy
- How the kubelet, the runtime and the CNI plugin create a pod network
- Pod CIDRs, node CIDRs and service CIDRs
- The three kinds of traffic: pod to pod, pod to service, outside to service

The model

The network model has a few simple rules. Every pod gets its own IP address. Pods on any node can reach pods on any other node using those addresses, without network address translation. Agents on a node, such as the kubelet, can reach all pods on that node. Containers inside one pod share a network namespace and talk over localhost. Services add a stable virtual IP on top, and are a separate layer.

How a pod gets its address

When the kubelet starts a pod, the container runtime creates a sandbox with a network namespace. The kubelet then calls the CNI plugin, a binary configured in `/etc/cni/net.d`, with the namespace and the pod identity. The plugin creates a virtual ethernet pair, puts one end in the pod and the other on the node, assigns an IP from the node pod range through IP address management, and sets up routes. When the pod is deleted, the plugin releases the address.

Address ranges

Three ranges must not overlap with each other or with your real networks. The pod CIDR is where pod IPs come from, and each node usually receives a slice, a /24 by default with kubeadm. The service CIDR is where ClusterIPs come from and exists only as rules on nodes. The node network carries node addresses. Overlap with an office or cloud range is a common cause of unreachable destinations.

```
kubectl get nodes -o custom-columns=NAME:.metadata.name,PODCIDR:.spec.podCIDR
kubectl cluster-info dump | grep -m1 service-cluster-ip-range
ls /etc/cni/net.d
```

Three kinds of traffic

Pod to pod traffic uses pod IPs directly and depends on the CNI. Pod to service traffic goes to a ClusterIP, which kube-proxy or an eBPF datapath rewrites to a pod IP. Traffic from outside enters through a NodePort, a load balancer or an ingress or gateway, and then reaches a service. Each kind has its own failure signature, so identify which one you are debugging first.

A quick address check

```
import ipaddress
pod = ipaddress.ip_network("10.244.0.0/16")
svc = ipaddress.ip_network("10.96.0.0/12")
node = ipaddress.ip_network("10.0.0.0/16")
print("pod/service overlap:", pod.overlaps(svc))
print("pod/node overlap:", pod.overlaps(node))
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: The Kubernetes network model: pods, nodes and the CNI

Part	What it does	What goes wrong
kubelet	Asks the runtime for a sandbox.	Runtime down.
Runtime	Creates the namespace.	Sandbox error.
CNI plugin	Wires and addresses.	No config file.
IPAM	Allocates the IP.	Range exhausted.
Pod network	Veth, IP and routes.	Overlap.

Flow: kubelet to Runtime / Runtime to CNI plugin / CNI plugin to IPAM / CNI plugin to Pod network

Incident: A pod range that overlapped the office network

Incident: *Pods could not reach an internal database after the cluster was built.*

```
$ kubectl exec web-1 -- curl -s -m 3 10.244.5.20:5432
curl: (28) Connection timed out after 3001 milliseconds
$ ip route get 10.244.5.20
10.244.5.20 via 10.244.0.1 dev cni0
(this is a simulated illustration, not a real incident)
```

The database lived in an office network that used the same range as the pod CIDR, so the node routed traffic for it to a local bridge instead of the office gateway. Rebuilding with a non-overlapping pod range was the real fix. Choose address ranges before installing, because changing them later is painful.

Common mistakes

- Choosing a pod CIDR that overlaps real networks
- Assuming Services have network interfaces
- Mixing up the service CIDR and the pod CIDR
- Installing two CNI plugins
- Forgetting that the CNI must exist before pods can start

Check yourself

Which rule belongs to the Kubernetes network model?

Answer: Pods can reach other pods on any node without NAT

What does the CNI plugin do?

Answer: Configures a pod network interface, address and routes

Where do ClusterIPs exist?

Answer: Only as rules on nodes, not on a network interface

Why avoid overlapping ranges?

Answer: Routing becomes ambiguous and some destinations become unreachable

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. CNI plugins: overlays, routing and eBPF

12 minutes. Compare the main CNI designs, overlay encapsulation, native routing and eBPF dataplanes, and choose one with the trade-offs in mind.

Several CNI plugins satisfy the same network model in different ways. They differ in performance, in the features they add and in how you debug them. A choice made at cluster creation is hard to undo, so it is worth understanding the families.

What you will learn

- Overlay and encapsulation
- Native routing with BGP or cloud routes
- eBPF dataplanes and what they replace
- Policy support and observability features
- How to choose

Overlay networks

An overlay wraps pod packets inside packets between nodes, commonly with VXLAN or Geneve. It works over almost any underlying network, because the nodes only need to reach each other. The cost is a small overhead per packet, a lower effective MTU and an extra layer to debug. Flannel with VXLAN is a simple example.

Native routing

A routed design gives pods addresses that the underlying network can route. Nodes advertise their pod ranges with BGP, as in Calico or Cilium in routing mode, or the cloud provider programs routes. There is no encapsulation, so performance is closer to the host network, and tools such as traceroute show the real path. It needs a network that cooperates, or cloud integration.

eBPF dataplanes

eBPF lets a plugin attach small verified programs to kernel hooks. Cilium uses it to implement forwarding, service load balancing and policy without long iptables chains, and can replace kube-proxy. Benefits are scale and visibility, with identity based policy and flow observability. The costs are kernel version requirements and a new way to debug.

Features to compare

Compare support for NetworkPolicy and any extensions, encryption between nodes, observability of flows, IPv6 and dual stack, multi-cluster connectivity, maximum cluster size and operational maturity. Check that the plugin supports your environment, for example a managed cloud with its own CNI.

```
kubectl get pods -n kube-system -o wide | grep -E "calico|cilium|flannel|weave"
ip -d link show | grep -E "vxlan|geneve" | head -3
ip route | head
```

Choosing

Start with requirements: network policy, scale, encryption, the base platform. A small lab can use a simple overlay. A production cluster usually wants policy support and good observability, which points to Calico or Cilium. Run the choice through a test in your environment before you commit, and plan how you will upgrade the plugin, since it runs on every node.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: CNI plugins: overlays, routing and eBPF

Part	What it does	What goes wrong
Overlay	VXLAN or Geneve.	MTU too large.
Native routing	BGP or cloud routes.	Network refuses routes.
eBPF dataplane	Kernel programs.	Old kernel.
Policy	Enforced by the plugin.	Plugin without support.
Observability	Flow visibility.	Not enabled.

Flow: Overlay to Native routing / Native routing to eBPF dataplane / eBPF dataplane to Policy / Policy to Observability

Incident: Large responses that hung

> **Incident:** *Small requests worked, but any response over a certain size stalled between pods on different nodes.*

```
$ kubectl exec web-1 -- ping -c1 -M do -s 1472 10.244.2.9
ping: local error: message too long, mtu=1450
$ ip link show vxlan.calico | grep mtu
mtu 1450
(this is a simulated illustration, not a real incident)
```

The overlay added headers, so the pod interface MTU was 1450, but the application tested paths with a larger size and relied on path MTU discovery that a firewall dropped. Small packets passed, and large ones stalled. Setting a consistent MTU across node and pod interfaces and allowing the relevant ICMP messages fixed it. Overlay overhead is a classic cause of "works for small requests" faults.

Common mistakes

- Ignoring MTU with overlays
- Choosing a plugin without policy support when you need policy
- Replacing the CNI on a running cluster casually
- Running an eBPF plugin on an unsupported kernel
- Not planning plugin upgrades

Check yourself

What does an overlay network do?

Answer: Encapsulates pod packets inside node to node packets

What is a trade-off of VXLAN?

Answer: Packet overhead and a lower effective MTU

What can an eBPF dataplane replace?

Answer: kube-proxy and long iptables chains

Why test a CNI choice before committing?

Answer: It runs on every node and is hard to change later

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Services: ClusterIP, NodePort and kube-proxy modes

12 minutes. See how a Service gives pods a stable address and how kube-proxy in iptables or IPVS mode implements load balancing, and what each Service type exposes.

Pods come and go and change IPs. A Service is the stable name and address in front of them. Understanding how a Service is turned into packet rules explains its behaviour, its limits and a good share of network faults.

What you will learn

- Service types and what each exposes
- How kube-proxy programs rules from Services and endpoints
- iptables, IPVS and nftables modes
- Session affinity, externalTrafficPolicy and source IPs
- Headless Services

Types

A ClusterIP Service is reachable only inside the cluster through a virtual IP. A NodePort adds a port, by default 30000 to 32767, on every node that forwards to the Service. A LoadBalancer asks the cloud provider for an external load balancer that targets the NodePorts. An ExternalName returns a DNS CNAME and does no proxying. A headless Service, with clusterIP None, has no virtual IP and returns the pod addresses in DNS.

kube-proxy

kube-proxy runs on every node, watches Services and EndpointSlices through the API server, and programs the node so that traffic to a ClusterIP and port is rewritten to a pod IP. It does not carry traffic itself in the common modes. In iptables mode it writes rules that select an endpoint at random, so large Services produce long rule lists. In IPVS mode it uses the kernel load balancer with hash tables, which scales better and offers more algorithms. Newer clusters can also use nftables mode.

Look at the rules

```
kubectl get svc web
kubectl get endpointslices -l kubernetes.io/service-name=web
sudo iptables -t nat -L KUBE-SERVICES -n | head
kubectl -n kube-system get cm kube-proxy -o jsonpath='{.data.config\.conf}' | grep -E "mode|ipvs"
```

Affinity and source IPs

By default each connection picks an endpoint independently. `sessionAffinity: ClientIP` keeps a client on the same pod for a period. For NodePort and LoadBalancer Services, `externalTrafficPolicy: Local` sends traffic only to pods on the receiving node and preserves the client source IP, but nodes without a local pod fail health checks. The default Cluster policy may hop to another node and replace the source IP.

Headless Services and stateful sets

A headless Service returns individual pod records, which is how StatefulSets give each pod a stable DNS name such as `db-0.db.prod.svc`. Clients that need to choose a member, such as a database driver, use these records.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Services: ClusterIP, NodePort and kube-proxy modes

Part	What it does	What goes wrong
Client pod	Calls the ClusterIP.	Wrong port.
kube-proxy rules	Rewrite to a pod IP.	Stale rules.
EndpointSlices	Ready pod addresses.	Empty list.
Backend pod	Handles the request.	Not ready.

Flow: Client pod to kube-proxy rules / kube-proxy rules to EndpointSlices / kube-proxy rules to Backend pod

Incident: A Service with no endpoints

Incident: *Calls to a Service hung, even though pods with the right name were running.*

```
$ kubectl get svc cart
NAME   TYPE        CLUSTER-IP     PORT(S)
cart   ClusterIP   10.96.44.18    80/TCP
$ kubectl get endpointslices -l kubernetes.io/service-name=cart
NAME         ADDRESSTYPE   PORTS   ENDPOINTS
cart-x7k2p   IPv4          80      <unset>
(this is a simulated illustration, not a real incident)
```

The Service selector said app=cart but the pods carried app=cart-api, so no endpoints matched and connections to the ClusterIP had nowhere to go. Correcting the selector filled the EndpointSlice. When a Service misbehaves, check endpoints before anything else.

Common mistakes

- Not checking endpoints first
- Selector and label mismatch
- Using NodePort where an ingress is safer
- Assuming the source IP is preserved
- Ignoring readiness probes that keep pods out of endpoints

Check yourself

What does a ClusterIP expose?

Answer: A virtual IP reachable inside the cluster

What does kube-proxy do in iptables mode?

Answer: Programs rules that rewrite Service addresses to pod addresses

What is the first check for a Service that does not respond?

Answer: Its endpoints

What does externalTrafficPolicy Local do?

Answer: Sends to pods on the receiving node and keeps the client IP

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. EndpointSlices and how traffic finds pods

11 minutes. Follow how readiness, selectors and the EndpointSlice controller decide which pods receive traffic, and why a ready pod may still get none.

Between a Service and a pod there is a list. That list, the EndpointSlice, is what really decides where traffic goes, and many apparent network problems are in fact wrong lists.

What you will learn

- Why EndpointSlices replaced Endpoints
- How the controller builds slices from selectors and readiness
- Terminating endpoints and graceful shutdown
- Topology aware routing
- Services without selectors

From Endpoints to slices

The old Endpoints object held every address of a Service in one resource, so a large Service made every update huge and sent it to every node. EndpointSlices split addresses into slices, by default up to 100 each, so an update changes one small object. kube-proxy and other consumers watch slices.

How slices are built

The EndpointSlice controller watches Services and pods. For each Service with a selector it lists matching pods and records their IP, port, node, zone and conditions. A pod is ready only if its containers pass readiness. Only ready endpoints receive new connections in normal operation.

Conditions

Each endpoint has ready, serving and terminating conditions. During a rolling update a pod that is shutting down becomes terminating, and stays serving until it exits if its readiness still passes, which helps drain connections. Pair this with a preStop delay and a termination grace period so that load balancers and kube-proxy have time to stop sending new traffic.

Zone aware routing

With topology aware routing, enabled by annotation or the traffic distribution field, a Service prefers endpoints in the same zone, which reduces latency and cross zone cost. It falls back when a zone has too few endpoints, so make sure replicas are spread across zones.

```
kubectl get endpointslice -l kubernetes.io/service-name=web -o yaml | head -40
kubectl get pods -l app=web -o wide
```

Selectorless Services

A Service without a selector gets no automatic slices. You create an EndpointSlice by hand to point at something outside the cluster, such as a database VM. It is also how some controllers publish backends, and a missing slice is the explanation for such Services failing.

Diagram: EndpointSlices and how traffic finds pods

Part	What it does	What goes wrong
Service selector	Matches labels.	Mismatch.
Slice controller	Builds the list.	Lagging.
EndpointSlice	Ready addresses.	Empty.
kube-proxy	Programs nodes.	Stale.
Ready pods	Receive traffic.	Not ready.

MARGA by EknathaLabs
marga.eknathalabs.com

Flow: Service selector to Slice controller / Slice controller to EndpointSlice / EndpointSlice to kube-proxy / kube-proxy to Ready pods

Incident: Errors during every rolling update

Incident: *Clients saw connection resets for a few seconds on each deploy.*

```
$ kubectl get deploy web -o jsonpath='{.spec.template.spec.terminationGracePeriodSeconds}'
30
$ kubectl get deploy web -o jsonpath='{.spec.template.spec.containers[0].lifecycle}'
(no output)
(this is a simulated illustration, not a real incident)
```

The container stopped accepting connections the moment it received SIGTERM, while kube-proxy and the load balancer still sent a few more. Adding a short preStop sleep gave the endpoint removal time to spread before shutdown began, and the resets stopped. Graceful shutdown is a network feature as well as an application one.

Common mistakes

- Ignoring readiness probes
- Stopping the listener immediately on SIGTERM
- Spreading replicas in one zone with zone routing on
- Creating Endpoints by hand instead of slices
- Assuming propagation of endpoint changes is instant

Check yourself

What does an EndpointSlice list?

Answer: Addresses and readiness for a Service's backends

Why were slices introduced?

Answer: Large Services made single Endpoints objects too big to update

How do you reduce resets during a rolling update?

Answer: Add a preStop delay and enough grace period

When does a Service need a manual EndpointSlice?

Answer: When it has no selector and points at an external backend

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. CoreDNS and service discovery

12 minutes. Understand how pods resolve names, how resolv.conf, ndots and search domains behave, how CoreDNS is configured, and how to tune and debug it.

Almost every request in a cluster starts with a DNS lookup. DNS faults look like network faults, and slow DNS looks like slow applications. A clear picture of how names resolve removes a lot of guesswork.

What you will learn

- Service and pod DNS names
- What is in a pod resolv.conf and why ndots matters
- The CoreDNS Corefile and plugins
- Scaling and caching
- Debugging resolution

Names

A Service named web in namespace shop has the name `web.shop.svc.cluster.local`. Pods in the same namespace can use just `web`, and from other namespaces `web.shop`. Headless Services also publish per pod names. SRV records carry ports. Pods have IP based names that are rarely used.

resolv.conf and ndots

The kubelet writes each pod resolv.conf with the cluster DNS Service IP as nameserver, a search list such as `shop.svc.cluster.local svc.cluster.local cluster.local`, and `options ndots:5`. A name with fewer than five dots is tried with each search suffix first, so a lookup for `api.example.com` makes several failing queries before the real one. Using a fully qualified name with a trailing dot, or lowering ndots in the pod spec, cuts that traffic.

The Corefile

CoreDNS runs as a Deployment in kube-system, behind the kube-dns Service. Its configuration is the Corefile in a ConfigMap, built from plugins. The kubernetes plugin answers cluster names from the API. The forward plugin sends other names to upstream resolvers. The cache plugin caches answers, and loop detects forwarding loops. You can add stub domains and custom zones.

```
kubectl -n kube-system get cm coredns -o jsonpath='{.data.Corefile}'
kubectl -n kube-system get deploy coredns
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=20
```

Scale and cache

Large clusters send heavy DNS traffic. Scale CoreDNS with replicas or autoscaling relative to cluster size, and consider NodeLocal DNSCache, a per node cache that reduces latency and conntrack pressure. Watch the request rate, cache hit ratio and error counts.

Debugging

Test from a throwaway pod rather than from the node, because the node resolver differs. Check the nameserver in resolv.conf, query a cluster name and an external name, and read CoreDNS logs.

```
kubectl run dnstest --image=busybox:1.36 --restart=Never --rm -it -- nslookup web.shop.svc.cluster \
    .local
kubectl exec web-1 -- cat /etc/resolv.conf
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: CoreDNS and service discovery

Part	What it does	What goes wrong
Pod resolver	Uses search and ndots.	Many wasted queries.
CoreDNS	Answers cluster names.	Overloaded.
API server	Source of Services.	Slow.
Upstream DNS	External names.	Unreachable.

Flow: Pod resolver to CoreDNS / CoreDNS to API server / CoreDNS to Upstream DNS

Incident: DNS slowness from search domains

Incident: *An application that called an external API had five second delays on its first call each minute.*

```
$ kubectl exec web-1 -- cat /etc/resolv.conf
search shop.svc.cluster.local svc.cluster.local cluster.local corp.example
nameserver 10.96.0.10
options ndots:5
$ kubectl -n kube-system logs -l k8s-app=kube-dns --tail=2
[ERROR] plugin/errors: 2 api.partner.com.corp.example. A: read udp 10.244.1.4:41872->10.0.0.2:53:  \
    i/o timeout
(this is a simulated illustration, not a real incident)
```

A lookup for api.partner.com with ndots 5 tried each search suffix first, and one of them, the corporate suffix, went to an upstream resolver that dropped queries. Each timeout added seconds. Using the fully qualified name with a trailing dot, and lowering ndots for that workload, removed the delay. Search domains are a hidden multiplier on DNS load.

Common mistakes

- Ignoring ndots and search domains
- Testing DNS from the node instead of a pod
- Leaving CoreDNS at two replicas in a large cluster
- Editing the Corefile without checking the loop plugin
- Blaming the network for resolution errors

Check yourself

What is the DNS name of Service web in namespace shop?

Answer: web.shop.svc.cluster.local

What does ndots:5 do?

Answer: Treats names with fewer than five dots as relative and tries search suffixes first

Where is CoreDNS configured?

Answer: In the Corefile in a ConfigMap

Where should you test DNS from?

Answer: A pod in the cluster

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Network policies in depth

13 minutes. Write network policies that select pods, namespaces and CIDRs, understand the additive and default deny model, and avoid the common traps such as DNS and the or-versus-and selector rule.

By default every pod can talk to every other pod. Network policies are the mechanism to limit that, but only if the CNI enforces them, and their selector rules are easy to get subtly wrong.

What you will learn

- The default allow behaviour and what changes when a pod is selected
- Ingress and egress rules and how they combine
- podSelector, namespaceSelector and ipBlock
- The classic mistake with two selectors in one rule
- Testing and patterns

Selection and the model

A NetworkPolicy selects pods with a podSelector. Pods that no policy selects stay open. As soon as a pod is selected by a policy that lists a policy type, the pod becomes isolated for that direction, and only traffic allowed by some policy is permitted. Policies are additive, so there is no deny rule: the union of all allow rules applies. An empty podSelector selects all pods in the namespace, which is how you write default deny.

Default deny and allow

Write a default deny for ingress, egress or both, then add allow policies for each flow. Remember DNS: with default deny egress, pods can no longer resolve names, so allow UDP and TCP port 53 to the DNS pods.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
```

Peers

A rule can name peers with podSelector for pods in the policy namespace, namespaceSelector for pods in matching namespaces, both together for specific pods in specific namespaces, or ipBlock for CIDR ranges with optional exceptions. Namespaces carry the label `kubernetes.io/metadata.name`, which makes selecting one by name easy.

MARGA by EknathaLabs marga.eknathalabs.com

One item or two

In a `from` list, two selectors in the same item must both match, an AND. Two separate list items are alternatives, an OR. A dash in the wrong place changes the meaning completely, often allowing far more than intended. Read the indentation of every rule.

```
kubectl get networkpolicy -n shop
kubectl describe networkpolicy allow-web -n shop
```

Testing and patterns

Test from a throwaway pod with the labels of the client you are testing. Check both an allowed and a denied flow. Common patterns: default deny per namespace, allow from the ingress controller namespace, allow to the database only from the app tier, and egress allow lists for external APIs. Remember that policies are only enforced if the CNI supports them, and that some policies act on node or host traffic differently.

Diagram: Network policies in depth

Part	What it does	What goes wrong
Policy	Selects pods.	Empty selector.
Ingress rules	Who may connect in.	Too wide.
Egress rules	Where pods may go.	DNS blocked.
CNI enforcement	Applies the rules.	Not supported.
Test pods	Prove allow and deny.	Not tested.

Flow: Policy to Ingress rules / Policy to Egress rules / Ingress rules to CNI enforcement / Egress rules to CNI enforcement / CNI enforcement to Test pods

Incident: One dash too many

Incident: *A policy intended to allow the frontend only ended up allowing every pod in the frontend namespace and every pod labelled web anywhere.*

```
$ kubectl get networkpolicy allow-web -n db -o yaml | grep -A8 "from:"
  - from:
    - namespaceSelector:
        matchLabels:
          team: frontend
    - podSelector:
        matchLabels:
          app: web
(this is a simulated illustration, not a real incident)
```

Two list items mean OR. The author wanted pods labelled app web in namespaces labelled team frontend, which needs both selectors in one item. The loose policy let any pod in the frontend namespace and any web pod in the database namespace connect. Correcting the indentation to a single item restored the intended AND. Review every from and to list for dashes.

Common mistakes

- Forgetting DNS in a default deny egress policy
- Using two list items where one item with two selectors is meant
- Expecting deny rules
- Using a CNI that does not enforce policy
- Not testing both allowed and denied flows

Check yourself

What does an empty podSelector select?

Answer: All pods in the namespace

MARGA by EknathaLabs marga.eknathalabs.com

How do policy rules combine?

Answer: As a union of allows

How do you require both a namespace and a pod label?

Answer: Put both selectors in one list item

What breaks after a default deny egress with no other rules?

Answer: DNS lookups

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Ingress and the Gateway API

12 minutes. Expose HTTP services with Ingress and the Gateway API, understand controllers, classes, routes and TLS, and know when each fits.

A Service gets traffic to pods. Ingress and the Gateway API take HTTP and HTTPS from outside the cluster and route it to the right Service by host and path. They are objects in the API, but they do nothing without a controller that implements them.

What you will learn

- Ingress objects, classes and controllers
- TLS termination and certificates
- Gateway, GatewayClass and routes
- Role separation and cross namespace routing
- Choosing between them and migration

Ingress

An Ingress declares host and path rules that map to Services, plus TLS settings. An ingress controller, such as ingress-nginx, Traefik or a cloud controller, watches Ingress objects and configures a proxy or load balancer. Which controller handles an object is chosen with `ingressClassName`. Without a controller, the Ingress is just data. Controller specific behaviour is added through annotations, which makes manifests less portable.

TLS

Ingress can terminate TLS using a certificate stored in a Secret of type kubernetes.io/tls, referenced in the `tls` section. Tools such as cert-manager request and renew certificates automatically. The proxy then forwards plain or re-encrypted traffic to the backends.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  ingressClassName: nginx
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

Gateway API

The Gateway API splits responsibilities across objects. A GatewayClass names a controller implementation, managed by the platform provider. A Gateway describes listeners, such as ports, protocols and TLS, and is owned by the cluster operator. Routes such as HTTPRoute attach to a Gateway and describe how requests reach Services, owned by application teams. The model supports header matching, traffic splitting, redirects and other features in the spec instead of annotations.

MARGA by EknathaLabs marga.eknathalabs.com

Sharing safely

Gateways can restrict which namespaces may attach routes, and cross namespace references need a ReferenceGrant. This lets a platform team run a shared Gateway while teams manage only their own routes, which Ingress could not express cleanly.

Choosing

Ingress is mature, widely supported and enough for simple host and path routing. The Gateway API is the direction of the project, with richer features and a better ownership model, and is the choice for new designs when your controller supports it. Both need a controller you operate and monitor. Check events and the status of the objects when traffic does not arrive.

```
kubectl get ingressclass
kubectl describe ingress web
kubectl get gateway,httproute -A
```

Diagram: Ingress and the Gateway API

Part	What it does	What goes wrong
Client	HTTPS request.	Wrong DNS.
Load balancer	Entry point.	Health check fails.
Controller and proxy	Implements the rules.	Missing class.
Ingress or HTTPRoute	Host and path rules.	Wrong backend.
Service and pods	Backends.	No endpoints.

Flow: Client to Load balancer / Load balancer to Controller and proxy / Controller and proxy to Ingress or HTTPRoute / Controller and proxy to Service and pods

Incident: An Ingress that nothing served

Incident: *A new Ingress showed no address for days.*

```
$ kubectl get ingress web
NAME   CLASS    HOSTS              ADDRESS   PORTS
web    <none>   shop.example.com             80
$ kubectl get ingressclass
NAME    CONTROLLER                  PARAMETERS
nginx   k8s.io/ingress-nginx        <none>
(this is a simulated illustration, not a real incident)
```

The Ingress had no class and there was no default class, so no controller claimed it. Setting ingressClassName to nginx made the controller pick it up and the address appeared. An Ingress with an empty address often just needs the right class, so check the class before debugging the proxy.

Common mistakes

- Creating an Ingress without a controller or class
- Relying on controller specific annotations for portability
- Forgetting that TLS Secrets must be in the same namespace
- Exposing a Service as NodePort instead of through a gateway
- Not checking the status of Gateway objects

Check yourself

What makes an Ingress object take effect?

Answer: A controller that implements it

What does ingressClassName select?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Which controller handles the Ingress

Who owns HTTPRoute objects in the Gateway API model?

Answer: Application teams

What allows a route to reference a Service in another namespace?

Answer: A ReferenceGrant

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Debugging Kubernetes networking

13 minutes. Use a fixed order to find network faults: identify the traffic type, test layer by layer, and use the right tools from inside the cluster.

Network faults are easy to chase in circles. A fixed method, working from the symptom to the layer, saves time and works in an exam as well as in production.

What you will learn

- Name the traffic type and the exact failing hop
- A layered checklist: DNS, Service, endpoints, pod, policy, node
- Tools: ephemeral debug containers, nicolaka netshoot, tcpdump, conntrack
- Reading conntrack and kube-proxy rules
- A worked sequence

Start with the question

Say precisely what fails: from which pod to which name and port, and what error you see. Connection refused means something answered with a reset, usually no listener. A timeout means packets are dropped or lost, so look at policy, routing or the endpoint being gone. Name resolution errors mean DNS. A TLS error is above the network.

The layered order

Test in this order and stop at the first failure. One, can the client resolve the name. Two, does the Service exist with the right port and selector. Three, does it have ready endpoints. Four, can you reach a pod IP directly, which bypasses the Service. Five, does a policy block the flow. Six, is the node level path healthy: routes, kube-proxy rules, conntrack and MTU.

```
kubectl exec client -- nslookup web.shop
kubectl get svc,endpointslices -n shop
kubectl exec client -- curl -s -m 3 http://10.244.2.9:8080/healthz
kubectl get networkpolicy -n shop
```

Tools in the cluster

A minimal image often has no tools. Use an ephemeral debug container to attach to a running pod, or run a toolbox pod such as netshoot with curl, dig, tcpdump and ss. Debug on the same node as the failing pod when node level behaviour matters.

```
kubectl debug -it web-1 --image=nicolaka/netshoot --target=app
kubectl run tools --image=nicolaka/netshoot --restart=Never -- sleep 3600
```

Node level checks

On the node, `ip route` shows pod routes, `ss -tnp` shows listeners, `iptables-save | grep KUBE-SVC` shows Service chains, and `conntrack -S` shows table statistics. A full conntrack table or insert failures cause random connection errors under load. UDP DNS traffic is a common victim.

A worked sequence

A pod times out calling `cart`. Name resolves, the Service has a ClusterIP, endpoints are empty, the selector says app=cart and the pods say app=cart-api. The fix is the selector. If endpoints had been present, the next test would have been a direct call to a pod IP, then policy, then the node. The method is the same each time.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Debugging Kubernetes networking

Part	What it does	What goes wrong
Symptom	Refused or timed out?	Vague.
DNS	Name resolves.	ndots, CoreDNS.
Service and endpoints	Ports and backends.	Selector.
Policy and routes	Allowed and routed.	Silent drops.
Node and conntrack	Rules and tables.	Table full.

Flow: Symptom to DNS / DNS to Service and endpoints / Service and endpoints to Policy and routes / Policy and routes to Node and conntrack

Incident: Random resets under load

Incident: *A busy service had random connection failures that no policy or Service change explained.*

```
$ sudo conntrack -S | head -3
cpu=0 found=912 invalid=3 insert=4410 insert_failed=1873 drop=1873
$ sudo sysctl net.netfilter.nf_conntrack_max net.netfilter.nf_conntrack_count
net.netfilter.nf_conntrack_max = 131072
net.netfilter.nf_conntrack_count = 131060
(this is a simulated illustration, not a real incident)
```

The connection tracking table was full, so new connections were dropped at random. Raising nf_conntrack_max, reducing short lived connections with keepalives and moving DNS to a node local cache fixed the faults. When symptoms are random and load dependent, check node level tables before changing application settings.

Common mistakes

- Skipping the Service and endpoint checks
- Testing only from a laptop
- Changing several things at once
- Assuming a timeout is the same as a refusal
- Ignoring node level limits such as conntrack

Check yourself

What does connection refused usually mean?

Answer: Something answered with a reset, often no listener on the port

What is a fast way to bypass a Service in a test?

Answer: Call a pod IP directly

How do you get tools into a pod with a minimal image?

Answer: Attach an ephemeral debug container

What can cause random connection drops under load?

Answer: A full conntrack table

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Kubernetes networking

Service debugging order, node commands, DNS settings and network policy reminders.

Check order for a failing Service

Step	Command
Service and selector	`kubectl get svc NAME -o wide`
Endpoints	`kubectl get endpointslices -l kubernetes.io/service-name=NAME`
Pod labels and readiness	`kubectl get pods --show-labels`
Direct pod IP	`kubectl exec POD -- curl -s POD_IP:PORT`
DNS	`kubectl exec POD -- nslookup NAME`
Policies	`kubectl get networkpolicy -A`

On the node

Job	Command	
kube-proxy mode	`kubectl -n kube-system logs ds/kube-proxy	head`	
iptables rules for a Service	`iptables -t nat -S	grep SERVICE`	
Conntrack entries	`conntrack -L	head`	
MTU	`ip link show`	

DNS

Setting	Effect
`ndots:5`	Short names try search domains first
Trailing dot	Skips the search list
NodeLocal DNSCache	Cache per node, fewer conntrack entries

Policy reminders

Rule	Why
Policies are additive	There are no deny rules
Allow DNS with default deny egress	Otherwise names fail
Same list item means AND	Separate items mean OR

MARGA by EknathaLabs marga.eknathalabs.com